

In perceptron, we change the weight & b after every epoch - while in perceptron the weights once defined is changed after every pass -

```
public class List extends ListFragment {
```

~~Array~~

@Override

```
public void onCreate(@Nullable Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);
```

```
    public ListFrag() {
```

```
        ArrayList<String> names = new ArrayList<>();
```

```
        names.add("String 1");
```

```
        names.add("String 2");
```

```
        names.add("String 3");
```

```
public interface ItemSelected() {
```

```
    void ItemSelected (int index);
```

```
}
```

context

```
new ArrayAdapter (getActivity(), R.layout.item,
```

```
ArrayAdapter<String> Adapter = new ArrayAdapter<> (names);
```

```
setListAdapter (Adapter);
```


How to manage an interface class? like

listfragment is called?

Itemselected ActivityParent class

Interface
Object

@Override

context of the parent Activity
public void onAttach(@NonNull Context context) {

super.onAttach(context);

ActivityParent = (Itemselected) context

↑
type casting of
parent Activity in
interface

}

@Override

list on which is clicked
public void onListItemSelected(@NonNull
ListView l, @NonNull View v, int pos,
int id)

view that tapped on screen
pos which is index of

super.onAttach(l, v, id, pos);

ActivityParent.onItemSelected(pos)

}

}

→ public class listfrag extends listfragment {
// members of the class


```
public class MainActivity extends AppCompatActivity  
implements ListView.OnItemClickListener {
```

```
@public void onItemClick (int index)  
{
```

```
tvDescription.setText ( phoneNumber.getText(index))  
}
```

Portrait and landscape modes - -

```
@Override
```

```
protected void onCreate ( Bundle savedInstanceState )  
{
```

```
super.onCreate ( savedInstanceState );
```

```
if ( findViewById ( R.id.layout_portrait ) != null )  
{
```

```
FragmentManager manager = getSupportFragmentManager  
manager ();
```

```
manager.beginTransaction()
```

```
show ( manager.findFragmentById ( R.id.layout_portrait )
```

```
hide ()
```

```
commit ()
```

```
}
```

```
show (
```


Fragment manager functions -

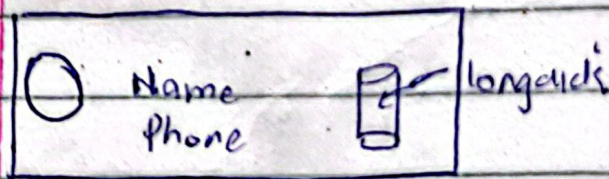
- ^{fragment}getSupportFragmentManager()
- beginTransaction()
- hide()
- getFragmentManager()
- commit()
- show()

```
getSupportFragmentManager().beginTransaction().  
hide(manager.getFragmentManager().findFragmentById(R.id.fragment)).commit();
```

→ Custom listview.

for only strings listview we use
built-in lists -

customized listview → we will design
our own listview with our own design -



1) First create model class -

2) Adapter extends ArrayAdapter <model class>

3) Data stored in arraylist -

model class:-

```
public class Contact {
```

```
    private String name, phone;
```

```
    public Contact();
```

```
    public Contact() {} // default constructor, getter  
    // setters
```

→ create a design(xml) file for the layout of the one card

MainActivity.java

Make hooks for editText, textView & FloatingActionButton

^{Action} FloatingButton fab;

ListView lvcontacts;

ContactAdapter adapter;

ArrayList<Contact> contacts;

^{set} fab.setOnClickListener (new setOnClickListener()
listener());

} @Override

void onClick (new View view) {

}


```
private void  
{
```

```
contactAdapter CA = new ContactAdapter (context, this);
```

```
public class ContactAdapter extends  
ArrayAdapter<Contacts> {  
{
```

```
public ContactAdapter (Context context,  
ArrayList<Contact> c)  
super (context, 0, objects);  
}
```

```
@NonNull
```

```
@Override
```

```
public View getView (int pos, View convertView,  
ViewGroup parent)  
{  
View V = convertView;  
if (V == null)  
{  
LayoutInflater inflater = (LayoutInflater) context.  
getSystemService (Context.LAYOUT_INFLATER_SERVICE);  
inflater (R.layout.item_contact, parent, false);  
}}
```



```
ivdelete.setOnClickListener (new View.OnClickListener() {
```

@Override

```
public void onClick (View v)
```

```
{  
    AlertDialog.Builder del del dialog =  
        new AlertDialog.Builder(context);  
    delDialog.setTitle("Confirmation");  
    delDialog.setMessage("Are You confirm to  
delete (Y/N)");
```

```
delDialog.setPositiveButton ("Delete", new  
DialogInterface.OnClickListener() {
```

@Override

```
public void onClick ( )  
{
```

```
    remove(c)
```

```
    notifyDataSetChanged ();
```

```
delDialog.setNegativeButton ("Cancel", new  
DialogInterface.OnClickListener() {
```

@Override

```
public void onClick (Dialog
```

```
    }
```

```
}
```



```
View.setOnLongClickListener (new View.OnLong  
ClickListener() {
```

@ override

```
public boolean onLongClick (View V) {
```

```
AlertDialog.Builder b = new AlertDialog.  
Builder (context);
```

```
b.setTitle ("update user Data");
```

~~b.setMessage~~

```
View view = LayoutInflater.from (context)  
.inflate (R.layout."filename", null,  
false);
```

```
updateDialog.setView (view);
```

L.T Name = ^{view.}getText (findViewById (R.id.etName));

L.T Pro = ^{view.}getText (findViewById (R.id.etPhone));

```
updateDialog.setPositiveButton (DialogInterface.  
dialog, int which)  
etName.
```

```
String name = getText ().toString().trim();
```

```
String phone = etPhone.getText ().toString().trim();
```

```
if (name.isEmpty() || phone.isEmpty())
```

?

```
Toast.makeText (context, "something is  
missing", Toast.LENGTH_SHORT
```

diag.

```
notifn.put (etName, name);
```


Both we use (MyApplication) | public static method mail
 public void onCreate mails;
 { super.onCreate();
 mails = new ArrayList<>();
 Toast.makeText(context, old name + "is updated",
 Toast.LENGTH_SHORT);

updateDialog.show();

Have made a model class:

MainActivity.java

RecyclerView rvMails;

ArrayList<Mail> mails;
 init

private void init()

rvMails.setHasFixedSize(true); // make our app
 more efficient

adapter = new MailAdapter(this, my application, mails);

rvMails.setLayoutManager(new LinearLayoutManager
 (this, LinearLayoutManager
 .Horizontal, true));

LayoutManager
 rvMails.setLay = new GridLayoutManager(this, 2);

rvMails.
~~LayoutManager~~.setLayoutManager(layout);

rvMails.setAdapter(adapter);

}

notifyItemChanged(position) ← whenever the
 record is changed

ViewHolder >

{

public MailAdapter (Context context, ~~Array~~
<mail> mails)

{

this.context = context;

this.mails = mails;

}

@NonNull

@Override

public class ^{Mail} ViewHolder extends RecyclerView
ViewHolder

public MailViewHolder (@NonNull itemView)
{
super(itemView);

// Just bind the views in it >

}

}

@Override

public void onBindViewHolder (@NonNull
MailViewHolder holder, int position)

{

list.get(position)

mail mail = ~~get(position)~~

~~it.get(position)~~ from

Holder.msg.setText(list.get(position));

Recycler view Adapter

// Hooks =

RecyclerView Rv;

~~At~~ MailAdapter mails;

FloatingActionButton fab;

public void Bundle saved

oncreate () { super.oncreate

init ();

^{setOn}fab.setOnClickListener () => {

addNewItem ();

};

~~private void~~ void init () addNewItem ();
}

myapplication.mails.add (newMail (" "));

~~adapter~~ =

Rv = findViewById (R.id.Rv);

~~Rv~~ mails = new mailAdapter (this, Application.mails)

Rv.setHasFixedSize (true); // does not change the

Rv.setLayoutManager

RecyclerView size even

~~Mail~~ set (new LinearLayoutManager

it items appeared or

Manager (this, ~~deleted~~)

LinearLayout Manager (Vertical, false));

Rv.mails.setAdapter (mails);


```
GridLayoutManager = new GridLayoutManager  
    (this, 2);
```

```
LinearLayoutManager = new LinearLayoutManager  
    (this, layoutManager.VERTICAL  
        false);
```

```
setLayoutManager()
```

```
8 V. setAdapter(adapter);
```

```
public class MailAdapter extends RecyclerView  
Adapter<MailAdapter.ViewHolder>
```

~~and~~

```
this class class MailViewHolder extends  
RecyclerView.ViewHolder {  
    // create Hooks
```

```
public class OnCreateViewHolder (new Item)  
{  
    super (ItemView);  
    tv = initialize Hooks;  
}
```

```
public OnCreateViewHolder (parent,  
    View view = LayoutInflater.from(context)  
        .inflate ( "filename", parent  
            , false);
```

```
return new MailViewHolder (view);
```


It is basically attaching the data of Mail to the
 public void onCreateViewHolder(@NonNull ^{XML files}
 ViewHolder VH, int position)
 {

~~VH~~ =

Mail mail = list.get(position);

VH.holder.setText(getText + mail());

Alert Dialog Boxes p21

AlertDialogBuilder = new AlertDialog.Builder(this);
 del.

mediator = new ^{TabLayout}TabLayoutMediator(viewPager, new
 TabLayoutMediator.TabConfigurationStrategy() {

@Override

public void onConfigureTab(@NonNull tab, int
 position)

{
 switch (pos)
 {

tab.setText("Text");

BadgeDrawable ^{Chats} bd = tab.onCreateBadge();

bd.charts.setNumber(100);

bd.charts.setImageDrawable(R.drawable.icon);

bd.charts.setMonChar('L');

break;

ta .prefetch() .commit()

mediator

new

Between MailViewHolder (view)

ViewPager2 . registerOnPageChange CallBack (new

ViewPager2 OnPageChange CallBack () }

@Override

public void onPageSelected (int pos) {

super .onPageSelected

if (isFirst)

}

BadgeDrawable = TabLayout .getTabAt (pos) .
create Badge()

bd .set number (0);

bd .set max character (2);

} else TabLayout .register create Badge

if (first == true; this;

FragmentManager {

// shared preferences this creator

SharedPreferences shared pref : getSharedPreferences
("name1", MODE

SharedPreferences .Editor editor
= shared .edit ()
= shared .commit ()

PRIVATE

editor .putString ("no

apply () -

editor .apply -

clear ()



implements

implements ~~nav~~ NavigationView.OnNavigationItemSelectedListener {

DrawerLayout drawerLayout; ActionBar actionBar;
ToolBar toolbar; NavigationView navView;

ActionBar

setSupportActionBar(toolbar); // ignore the ugly
ActionBar

toggle = new ActionBarDrawerToggle(this, ~~ActionBar~~ ^{toolbar}, drawerLayout,
toolbar, R.string.open, R.string.close);
drawerLayout.addDrawerListener(toggle);
toggle.syncState();

navView.setNavigationItemSelectedListener(this);

if (savedInstanceState == null)

{

load fragment (new Home Frag

onOptionsItemSelected (MenuItem item)

pos item

if